INDEX

SER	DESCRIPTION	PAGE NO
01	INTRODUCTION	04
02	OBJECTIVES OF THE PROJECT	05
03	PROPOSED SYSTEM	06
04	SYSTEM DEVELOPMENT LIFE CYCLE (SDLC)	08
05	PHASES OF SYSTEM DEVELOPMENT LIFE CYCLE	09
06	FLOW CHART	16
07	SOURCE CODE	18
08	OUTPUT	25
09	TESTING	31
10	HARDWARE AND SOFTWARE REQUIREMENTS	34
11	INSTALLATION PROCEDURE	35
12	BIBLIOGRAPHY	36

01. INTRODUCTION

In the contemporary financial landscape, the modernization and digitization of commercial operations have transitioned from an operational convenience to an imperative necessity. The "BANK MANAGEMENT SYSTEM" is a dedicated database-backed software application engineered to handle, automate, and streamline everyday branch-level accounting and administrative responsibilities.

For decades, financial establishments operated via manual physical ledgers, ink-and-paper journals, and paper registers. While traditionally functional, these legacy mechanisms suffer from severe limitations: human calculation fatigue, mathematical discrepancies, catastrophic physical vulnerability to fire or water, and extreme labor delays during routine account reconciliations. Locating a single historical transaction often demanded digging through hundreds of archived binders.

The emerging paradigm of digital information systems makes comprehensive customer and transaction intelligence accessible instantaneously. By deploying a computerized architecture, customer account opening, verification, deposits, withdrawals, and ledger adjustments are completed within fractions of a second.

CORE PILLARS OF THE SYSTEM

- Operational Efficiency: Eliminating customer queue congestion by enabling high-speed automated credit and debit operations directly at the teller console.
- Data Security & User Access Control: Restricting system access to authorized operators through unique operator IDs and mandatory 4-digit PIN authentication.
- Referential Integrity: Synchronizing customer profiles with append-only transaction audit logs to guarantee that every financial debit or credit is permanently logged with an automated date timestamp.
- Real-Time Reporting: Providing immediate terminal reports on customer account balances and full historical account statements without manual calculation.

Developed utilizing Python 3.12 and MySQL Database Management System, this project exemplifies the application of core computer science fundamentals to an authentic commercial requirement, serving as an extensible prototype for computerized financial administration.

02. OBJECTIVES OF THE PROJECT

The fundamental objective of this project is to bridge theoretical computer science pedagogy with real-world software engineering, equipping students with practical software design, database interfacing, and system validation skills.

1. TECHNICAL OBJECTIVES

- Relational Database Architecture: Design and implement a normalized database schema in MySQL comprising distinct entity tables (user_table, customer_details, and transactions) with primary key and foreign key relationships.
- Python-MySQL Connectivity: Establish reliable client-server communication using Python's standard mysql.connector library to execute dynamic Data Definition (DDL) and Data Manipulation (DML) queries.
- Clean Procedural Programming: Construct modular, maintainable, and readable procedural code strictly conforming to the CBSE Class 12 Computer Science syllabus, utilizing clean functions and standard library modules like datetime.
- Overdraft & Fraud Prevention: Implement rigorous programmatic validation checks to prevent negative account balances by rejecting withdrawal requests exceeding existing credit reserves.

2. OPERATIONAL & PEDAGOGICAL OBJECTIVES

- Automated Profile Management: Automate new customer onboarding, account number assignment, and residential detail cataloging.
- Atomic Transaction Logging: Guarantee ACID principles by issuing explicit database commit directives (conn.commit()) following every balance modification.
- Complete Auditability: Maintain an indelible, chronological transaction register detailing the transaction date, account identifier, withdrawal sum, and deposited sum for auditing purposes.
- Software Engineering Lifecycle Exposure: Demonstrate comprehensive grasp of the Software Development Life Cycle (SDLC) across initiation, concept analysis, structured design, coding, and empirical testing.

03. PROPOSED SYSTEM

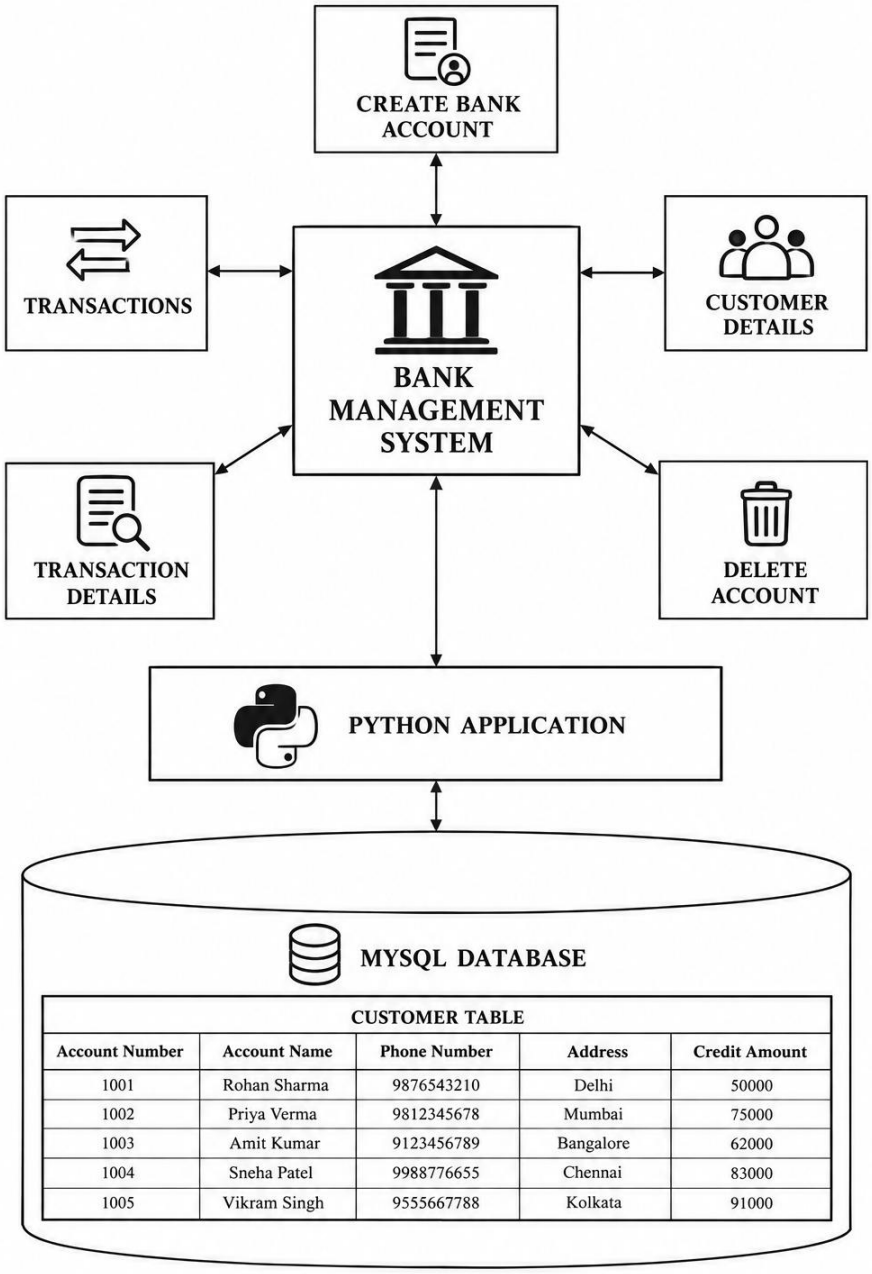
SHORTCOMINGS OF THE EXISTING MANUAL SYSTEM

Under traditional manual banking workflows, all customer transactions are transcribed into physical paper ledgers. This archaic operational model exhibits multiple critical points of failure: tellers calculating balances manually during peak business hours frequently introduce arithmetic errors that compromise institutional accounts; customer profiles must be transcribed across multiple register books, causing data drift and inconsistency; auditing past statements requires laborious manual searching through historical ledgers; and physical registers degrade over time and are completely defenseless against termites, dampness, fire hazards, and theft.

ARCHITECTURE & CAPABILITIES OF THE PROPOSED SYSTEM

The proposed computerized Bank Management System introduces an integrated digital environment that overcomes every manual bottleneck. Written in Python 3.12, the application interfaces with a high-performance local MySQL relational database engine. All transaction arithmetic is automated by the processor, completely eliminating human mathematical error. Account updates, profile lookups, and transaction logging happen synchronously in milliseconds.

The application's operational flow is organized around distinct, self-contained modules — authentication, account administration, transaction processing, and reporting — each rigorously isolated to prevent cascading failures. Every operator interaction is routed through a unified, menu-driven console that mirrors real-world teller workflows, so non-technical staff can adopt the system with minimal training. The architecture deliberately favors transparent, data-centric operations over complex abstractions, ensuring that each screen transition maps directly to a verifiable database action.



CUSTOMER TABLE

Account Number	Account Name	Phone Number	Address	Credit Amount
1001	Rohan Sharma	9876543210	Delhi	50000
1002	Priya Verma	9812345678	Mumbai	75000
1003	Amit Kumar	9123456789	Bangalore	62000
1004	Sneha Patel	9988776655	Chennai	83000
1005	Vikram Singh	9555667788	Kolkata	91000

Figure 3.1: Bank Management System Architecture Diagram

04. SYSTEM DEVELOPMENT LIFE CYCLE (SDLC)

The Systems Development Life Cycle (SDLC) is a disciplined project management methodology that partitions software engineering endeavors into well-defined, verifiable phases. Segmenting development allows engineers and project supervisors to validate requirements, assess risks, and verify milestones before moving to subsequent development stages.

Applying SDLC principles ensures that the Bank Management System achieves high reliability, data integrity, and strict adherence to user specifications. The complete sequential workflow followed throughout the lifecycle of this project is presented in the flowchart below:

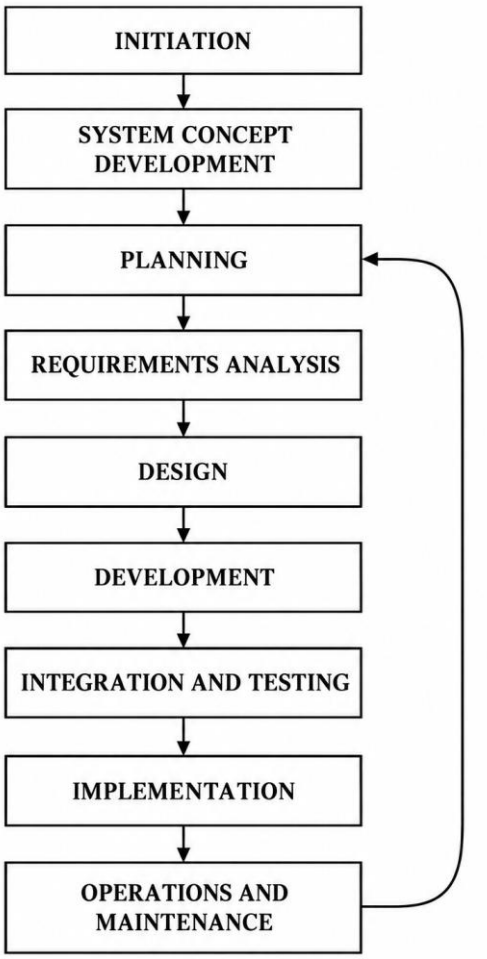


Figure 4.1: Software Development Life Cycle (SDLC) Flowchart

05. PHASES OF SYSTEM DEVELOPMENT LIFE CYCLE

PHASE 1: INITIATION PHASE

The Initiation Phase marks the official start of the project. It establishes whether an automated software system is warranted to replace manual banking logs and identifies the high-level operational boundaries.

Objectives: Key objectives accomplished during this phase include:

- Identifying the primary business problem: delays and mathematical inaccuracies inherent in paper passbook updating.
- Establishing initial feasibility: confirming that Python and MySQL provide a cost-effective, robust software stack for local branch banking.
- Defining project scope: restricting operations to account registration, customer profiling, deposit/withdrawal transactions, and account lifecycle management.
- Appointing project roles and establishing mentorship milestones under guidance.

PHASE 2: SYSTEM CONCEPT DEVELOPMENT PHASE

During this phase, the abstract project concept is refined into a defined operational model. The system boundaries, stakeholder interactions, and operational environments are documented.

Key Activities: Key activities performed in this phase:

- Defining system boundary: establishing local command-line interface interactions with local MySQL persistence.
- Identifying user classes: bank tellers, account supervisors, and registered customers.
- Assessing project risks: identifying risks such as negative account balances, unhandled database disconnections, and SQL syntax errors, and formulating mitigation strategies.
- Formulating performance criteria: ensuring response times under 1 second for standard account queries.

PHASE 3: PLANNING PHASE

The Planning Phase transforms the project concept into a comprehensive work breakdown structure. Timelines, resource allocations, and coding deliverables are scheduled to ensure systematic project execution.

Deliverables: Key deliverables of this phase:

- Development Schedule: allocating specific milestones for database schema definition, Python script authoring, test case creation, and report compilation.
- Resource Planning: identifying hardware prerequisites (PC, keyboard, monitor) and software prerequisites (Python 3.12 interpreter, MySQL Server 8.0, mysql-connector-python package).
- Quality Management Plan: establishing validation standards for inputs (e.g. 4-digit numeric passwords, positive transaction amounts).

PHASE 4: REQUIREMENTS ANALYSIS PHASE

Requirements analysis involves formally gathering, analyzing, and documenting what the system must accomplish from a functional and technical standpoint.

1. Functional Requirements:

- User Authentication: The system must enforce username and 4-digit PIN verification before granting access.
- Account Registration: The system must record unique account numbers, account holder names, 10-digit phone numbers, addresses, and initial deposit amounts.
- Transaction Processing: The system must process deposits and withdrawals, updating account credit balances immediately and recording an entry in the transaction log.
- Overdraft Prevention: The system must reject withdrawal requests exceeding the current credit balance.
- Account Inquiries: The system must retrieve full profile details and chronological transaction logs on demand.

2. Non-Functional Requirements:

- Data Integrity: Ensuring referential consistency and ACID properties during database write operations.
- Usability: Clear, numbered command-line menu navigation with informative confirmation messages.

PHASE 5: SYSTEM DESIGN PHASE

The System Design Phase translates documented requirements into technical architectural blueprints. Design activities focus on modular decomposition, procedural control flow, and data structuring.

1. Modular Architecture:

- Authentication Module (main_bank.py): Handles user registration, credentials storage, PIN validation, and initial routing.
- Banking Operations Module (menu.py): Manages the primary operational loop, providing 6 core functions: account creation, transaction execution, detail inquiry, statement listing, account termination, and session exit.
- Database Schema Definition Modules (table.py, user_table.py, transaction_table.py): Independent Data Definition Language (DDL) scripts to initialize required database tables.

2. Data Flow & Control Logic:

Control flow follows a hierarchical menu structure. Upon launching the application, the user registers or authenticates. Upon successful authentication, the control transfers to main_menu(), which iterates continuously until the user explicitly selects option 6 (Quit). Every transaction commits changes immediately to prevent data inconsistency.

3. Error Handling Strategy:

- Account Existence Checks: Verifying that account numbers exist prior to executing transactions or lookups.
- Sufficient Funds Check: Ensuring account balance is strictly greater than or equal to the requested withdrawal amount before executing SQL updates.

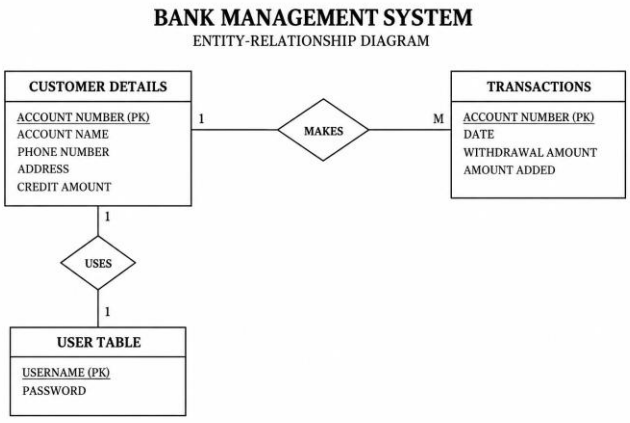


Figure 5.1: Bank Management System Entity-Relationship (ER) Diagram

DATABASE DESIGN & ENTITY-RELATIONSHIP (ER) MODELING

The relational database schema is structured into three normalized tables designed to minimize redundancy while preserving data relationships. The database is titled 'bank' in MySQL.

• Table 1: user_table — Stores operator authentication credentials.

- username VARCHAR(25) PRIMARY KEY — Unique username for system access.
- passwrd VARCHAR(25) NOT NULL — 4-digit security PIN.

• Table 2: customer_details — Stores profile and financial state of account holders.

- acct_no INT PRIMARY KEY — Unique primary key identifying the bank account.
- acct_name VARCHAR(25) — Full legal name of the account holder.
- phone_no BIGINT — Customer contact number.
- address VARCHAR(50) — Place / residential city.
- cr_amt FLOAT — Current balance credit amount.

• Table 3: transactions — Stores an append-only audit trail of financial operations.

- acct_no INT — Account number associated with the transaction (Foreign Key relationship).
- date DATE — Date timestamp of transaction execution.
- withdrawal_amt FLOAT — Amount debited (0 if deposit).
- amount_added FLOAT — Amount credited (0 if withdrawal).

PHASE 6: SOFTWARE DEVELOPMENT PHASE (CODING)

The Development Phase entails translating architectural designs and database schemas into clean, executable Python 3.12 source code. Development strictly adhered to CBSE Class 12 Computer Science programming conventions.

Implementation Highlights: Coding Guidelines & Principles Followed:

- Clean Python 3.12 Syntax: Replaced deprecated string concatenations and awkward spacing with clear, readable f-strings (f"...").
- Modular Procedural Design: Encapsulated banking logic inside clean functions (e.g. main_menu()) equipped with an __name__ == '__main__' execution block for flexible invocation.
- Correct SQL Transaction Control: Utilized explicit conn.commit() invocations after every INSERT, UPDATE, and DELETE query to ensure changes are persistently written to disk.
- Loop Integrity: Implemented a robust while True loop with an explicit exit condition, resolving the previous infinite-loop flaw present in older scripts.
- Standard Library Utilization: Leveraged Python's built-in datetime module for automated timestamping of financial ledger entries.
- CBSE Class 12 Alignment: Avoided excessive abstractions, classes/OOP, or external frameworks, keeping the codebase fully aligned with CBSE Class 12 evaluation criteria.

PHASE 7: INTEGRATION AND TESTING PHASE

During this phase, individual software modules were integrated and tested as a unified software system. Both Python client scripts and the local MySQL server instance were exercised across standard and boundary conditions.

Testing Workflow: Testing Stages Conducted:

- Unit Testing: Verifying individual SQL operations independently (e.g., verifying that INSERT INTO user_table accurately populates rows in user_table).
- Integration Testing: Testing the handshake between main_bank.py and menu.py to ensure that user credentials pass seamlessly into the operational dashboard.
- Boundary Condition Testing: Testing boundary values, such as depositing Rs. 0, withdrawing an amount equal to the exact balance, and attempting to withdraw Rs. 1 more than the current balance.
- Invalid Input Testing: Verifying system resilience when invalid account numbers are submitted for transactions or lookups.
- Database Synchronization Testing: Validating that executing an account deletion removes the record from both customer_details and the transactions log.

PHASE 8: IMPLEMENTATION PHASE

The Implementation Phase transitions the tested software system into operational readiness. It involves initializing the physical hardware environment, establishing database instances, and validating operational workflows.

Milestones: Deployment Milestones Accomplished:

- MySQL Database Initialization: Created the dedicated bank relational schema on localhost.
- DDL Script Execution: Executed table.py, user_table.py, and transaction_table.py to create required relational structures.
- Environment Verification: Tested connectivity between the Python 3.12 interpreter and the local MySQL 8.0 server using root credentials.
- Initial Seeding: Created administrative and sample customer records to confirm operational health.

PHASE 9: OPERATIONS AND MAINTENANCE PHASE

The Operations and Maintenance Phase encompasses post-deployment monitoring, ongoing data integrity verification, and planning future software enhancements.

Operational Policies: Maintenance Best Practices:

- Database Backups: Routine export of MySQL database tables (e.g., via mysqldump) to safeguard against accidental storage failure.
- Integrity Auditing: Periodic verification of transaction logs against customer current credit balances to ensure financial reconciliation.
- System Evolution: Planning future upgrades, such as graphical user interface (GUI) integration using Tkinter and interest rate calculation modules.

06. FLOW CHART

OVERALL WORKING PROCESS OF BANK MANAGEMENT SYSTEM

The comprehensive operational workflow of the Bank Management System is diagrammed below. It traces the lifecycle of user interaction from system startup, through operator registration or PIN authentication, into the primary 6-option command dashboard, and through subsequent database synchronization and reporting.

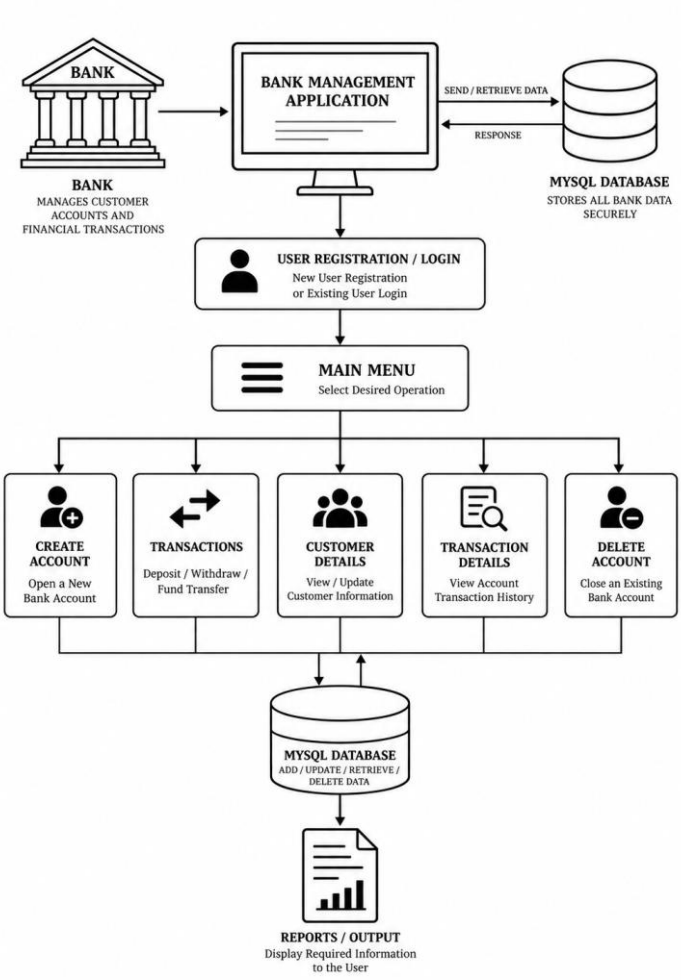


Figure 6.1: Overall Working Process of Bank Management System

BANK TRANSACTION PROCESS FLOWCHART

The dedicated flowchart below details the precise operational logic governing financial transactions (Cash Withdrawals and Cash Deposits). It highlights validation steps including account verification, choice branching, overdraft prevention checks, ledger updates, and transaction logging.

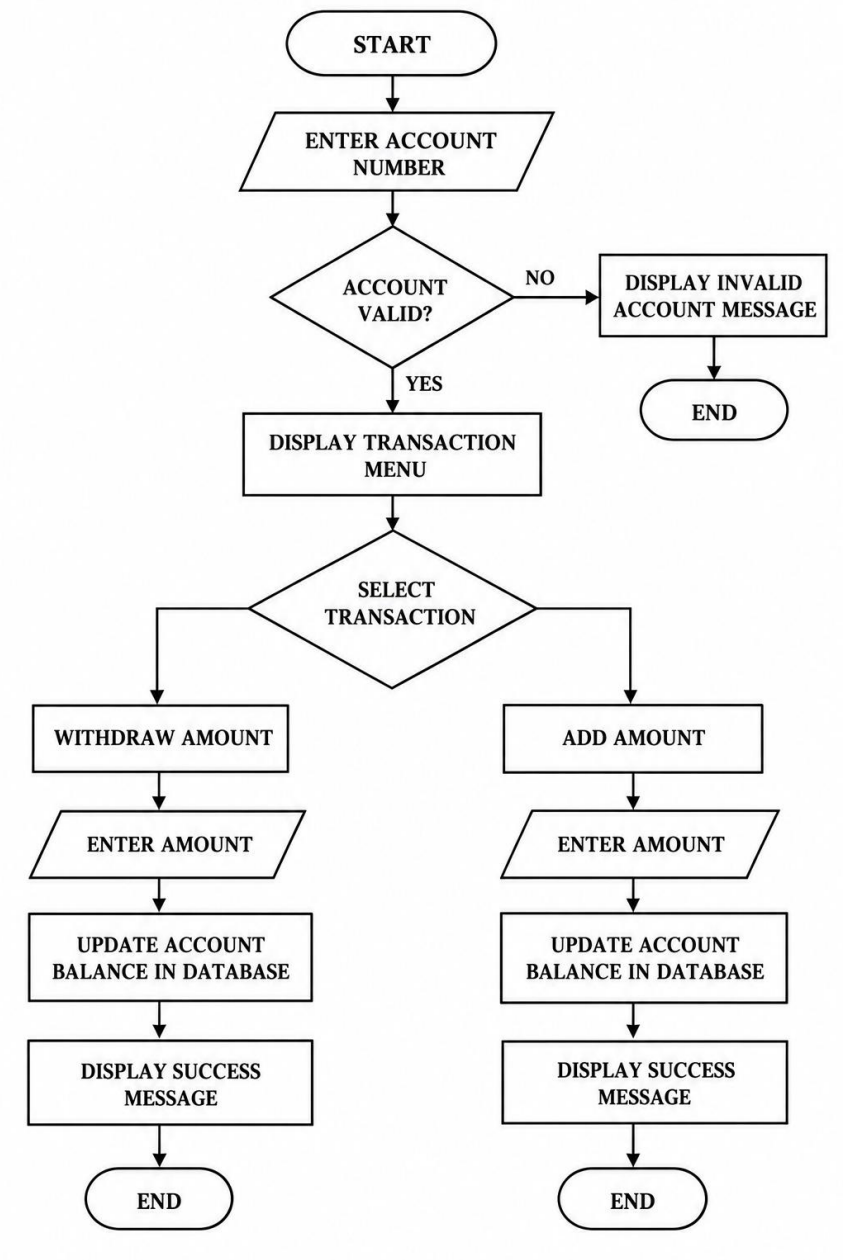


Figure 6.2: Bank Transaction Process Flowchart

07. SOURCE CODE

1. DATABASE INITIALIZATION SCRIPTS (DDL)

The following three standalone scripts establish the relational database tables in the MySQL 'bank' database.

A. customer_details Table Setup (table.py):

```
# table.py - Database setup for customer_details table
import mysql.connector as sql

conn = sql.connect(host='localhost', user='root', passwd='manager', database='bank')
cur = conn.cursor()

cur.execute('''
    CREATE TABLE IF NOT EXISTS customer_details (
        acct_no INT PRIMARY KEY,
        acct_name VARCHAR(25),
        phone_no BIGINT,
        address VARCHAR(50),
        cr_amt FLOAT
    )
''')
conn.commit()
print("Table 'customer_details' created successfully.")
```

B. user_table Table Setup (user_table.py):

```
# user_table.py - Database setup for user authentication
import mysql.connector as sql

conn = sql.connect(host='localhost', user='root', passwd='manager', database='bank')
cur = conn.cursor()

cur.execute('''
    CREATE TABLE IF NOT EXISTS user_table (
        username VARCHAR(25) PRIMARY KEY,
        passwrd VARCHAR(25) NOT NULL
    )
''')
conn.commit()
print("Table 'user_table' created successfully.")
```

C. transactions Table Setup (transaction_table.py):

```
# transaction_table.py - Database setup for transaction audit logs
import mysql.connector as sql

conn = sql.connect(host='localhost', user='root', passwd='manager', database='bank')
cur = conn.cursor()

cur.execute('''
    CREATE TABLE IF NOT EXISTS transactions (
        acct_no INT,
        date DATE,
        withdrawal_amt FLOAT,
        amount_added FLOAT
    )
''')
conn.commit()
print("Table 'transactions' created successfully.")
```

2. MAIN SYSTEM ENTRY SCRIPT (main_bank.py - Part 1)

The main_bank.py script acts as the master authentication gatekeeper and security boundary for the Bank Management System. Upon execution, the module initializes runtime dependencies, connects to the local MySQL database instance using host, user, and password parameters, and initializes an operational cursor for transactional communication.

Part 1 implements the application bootstrapper and the operator onboarding sequence. It presents an interactive terminal interface that timestamps system entry using Python's standard datetime module. When Option 1 (Register) is selected, the routine prompts the operator for a new username and a mandatory 4-digit security PIN. It validates the credentials and executes an INSERT query against the user_table relation, followed by an immediate conn.commit() to ensure persistent ACID storage before launching the main banking menu.

```
# main_bank.py - User Authentication and System Gateway
import datetime as dt
import mysql.connector as sql
import menu

# Establishing connection with MySQL server
conn = sql.connect(host='localhost', user='root', passwd='manager', database='bank')
cur = conn.cursor()

print('========================= WELCOME TO STARK BANK =========================')
print(dt.datetime.now())
print('1. REGISTER')
print('2. LOGIN')
print('=========================================================================')
print()

n = int(input('enter your choice = '))
print()

# Option 1: New User Registration
if n == 1:
    name = input('Enter a Username = ')
    passwd = input('Enter a 4 DIGIT Password = ')

    # Inserting credentials into user_table
    query = f"INSERT INTO user_table (username, passwrd) VALUES ('{name}', '{passwd}')"
    cur.execute(query)
    conn.commit()
    print()
    print('USER created successfully!')
    print()

    # Launch main banking operations menu
    menu.main_menu()
```

2. MAIN SYSTEM ENTRY SCRIPT (main_bank.py - Part 2)

Continuation of main_bank.py detailing user authentication and dashboard redirection.

```
# Option 2: Existing User Login Authentication
elif n == 2:
    name = input('Enter your Username = ')
    passwd = input('Enter your 4 DIGIT Password = ')

    # Verifying credentials against database
    query = f"SELECT * FROM user_table WHERE username = '{name}' AND passwrd =
'{passwd}'"
    cur.execute(query)

    if cur.fetchone() is None:
        print()
        print('Invalid username or password')
    else:
        print()
        print('Login successful!')
        print()
        # Direct authenticated user to main dashboard
        menu.main_menu()
else:
    print('Invalid choice! Please select 1 or 2.')
```

3. BANKING OPERATIONS MODULE (menu.py - Part 1)

The menu.py module handles all core banking features. Part 1 covers initialization and Account Creation. Apart from the creation routine itself, this first part establishes the shared database cursor and the master dispatch loop that governs every subsequent operator choice, giving the module a single, predictable entry point.

```
# menu.py - Core Banking Operations Dashboard
import datetime as dt
import mysql.connector as sql

conn = sql.connect(host='localhost', user='root', passwd='manager', database='bank')
cur = conn.cursor()

def main_menu():
    while True:
        print()
        print('======================== MAIN MENU ========================')
        print('1. CREATE BANK ACCOUNT')
        print('2. TRANSACTION')
        print('3. CUSTOMER DETAILS')
        print('4. TRANSACTION DETAILS')
        print('5. DELETE ACCOUNT')
        print('6. QUIT')
        print('===========================================================')
        print()

        n = int(input('Enter your CHOICE = '))
        print()

        # Feature 1: Create New Bank Account
        if n == 1:
            acc_no = int(input('Enter your ACCOUNT NUMBER = '))
            acc_name = input('Enter your ACCOUNT NAME = ')
            ph_no = int(input('Enter your PHONE NUMBER = '))
            add = input('Enter your place = ')
            cr_amt = float(input('Enter your credit amount = '))

            query = f"INSERT INTO customer_details VALUES ({acc_no}, '{acc_name}',
{ph_no}, '{add}', {cr_amt})"
            cur.execute(query)
            conn.commit()
            print()
            print('Account Created Successfully!!!!!')
```

3. BANKING OPERATIONS MODULE (menu.py - Part 2)

Continuation of menu.py detailing the financial Transaction Processing engine.

```
        # Feature 2: Financial Transactions (Withdrawal & Deposit)
        elif n == 2:
            acct_no = int(input('Enter Your Account Number = '))
            cur.execute(f"SELECT * FROM customer_details WHERE acct_no = {acct_no}")
            data = cur.fetchall()

            if len(data) == 0:
                print()
                print('Account Number Invalid. Please try again.')
            else:
                current_bal = data[0][4]
                print()
                print('1. WITHDRAW AMOUNT')
                print('2. ADD AMOUNT')
                print()
                x = int(input('Enter your CHOICE = '))
                print()
                today_date = dt.date.today()

                # Sub-option 1: Cash Withdrawal
                if x == 1:
                    amt = float(input('Enter withdrawal amount = '))
                    if amt > current_bal:
                        print()
                        print('Insufficient Balance!')
                    else:
                        cur.execute(f"UPDATE customer_details SET cr_amt = cr_amt -
{amt} WHERE acct_no = {acct_no}")
                        cur.execute(f"INSERT INTO transactions VALUES ({acct_no},
'{today_date}', {amt}, 0)")
                        conn.commit()
                        print()
                        print('Account Updated Successfully!!!!!')

                # Sub-option 2: Cash Deposit
                elif x == 2:
                    amt = float(input('Enter amount to be added = '))
                    cur.execute(f"UPDATE customer_details SET cr_amt = cr_amt + {amt}
WHERE acct_no = {acct_no}")
                    cur.execute(f"INSERT INTO transactions VALUES ({acct_no},
'{today_date}', 0, {amt})")
                    conn.commit()
                    print()
                    print('Account Updated Successfully!!!!!')
```

3. BANKING OPERATIONS MODULE (menu.py - Part 3)

Continuation of menu.py covering Customer Inquiries, Transaction Statement Logs, Account Termination, and Session Exit.

```
        # Feature 3: View Customer Details
        elif n == 3:
            acct_no = int(input('Enter your account number = '))
            print()
            cur.execute(f"SELECT * FROM customer_details WHERE acct_no = {acct_no}")
            data = cur.fetchall()

            if len(data) == 0:
                print('Invalid Account number')
            else:
                for row in data:
                    print('ACCOUNT NO      =', row[0])
                    print('ACCOUNT NAME    =', row[1])
                    print('PHONE NUMBER    =', row[2])
                    print('ADDRESS         =', row[3])
                    print('CURRENT BALANCE =', row[4])
                    print()

        # Feature 4: View Transaction Statements
        elif n == 4:
            acct_no = int(input('Enter your account number = '))
            print()
            cur.execute(f"SELECT * FROM customer_details WHERE acct_no = {acct_no}")
            if cur.fetchone() is None:
                print('Invalid Account number')
            else:
                cur.execute(f"SELECT * FROM transactions WHERE acct_no = {acct_no}")
                data = cur.fetchall()
                if len(data) == 0:
                    print('No transactions found for this account.')
                else:
                    print(f"{'ACC NO':<10} {'DATE':<15} {'WITHDRAWAL':<15} {'DEPOSIT':<15}")
                    print('-' * 55)
                    for row in data:
                        print(f"{row[0]:<10} {str(row[1]):<15} {row[2]:<15} {row[3]:<15}")
                    print()

        # Feature 5: Delete Account
        elif n == 5:
            print('DELETE YOUR ACCOUNT')
            acct_no = int(input('Enter your account number = '))
            cur.execute(f"SELECT * FROM customer_details WHERE acct_no = {acct_no}")
            if cur.fetchone() is None:
                print('Account does not exist!')
            else:
                cur.execute(f"DELETE FROM customer_details WHERE acct_no = {acct_no}")
                cur.execute(f"DELETE FROM transactions WHERE acct_no = {acct_no}")
                conn.commit()
                print()
                print('ACCOUNT DELETED SUCCESSFULLY')

        # Feature 6: Quit Application
        elif n == 6:
            c = input('DO YOU WANT TO EXIT (y/n) = ')
            if c.lower() == 'y':
                print()
                print('THANK YOU PLEASE VISIT AGAIN')
                break

if __name__ == '__main__':
    main_menu()
```

08. OUTPUT

OUTPUT SCREEN 1: SYSTEM WELCOME & USER REGISTRATION

Upon running main_bank.py, the user is presented with the Stark Bank header, current timestamp, and entry options. Selecting Option 1 allows the registration of a new operator.

```
======================== WELCOME TO STARK BANK ========================
2026-10-26 10:15:32.418291
1. REGISTER
2. LOGIN
=======================================================================

enter your choice = 1

Enter a Username = riddhiman
Enter a 4 DIGIT Password = 1298

USER created successfully!
```

OUTPUT SCREEN 2: USER LOGIN & MAIN MENU DASHBOARD

Selecting Option 2 prompts the operator for their username and 4-digit security PIN. Upon successful verification against user_table, the system unlocks the interactive Main Menu.

```
======================== WELCOME TO STARK BANK ========================
2026-10-26 10:18:04.120954
1. REGISTER
2. LOGIN
=======================================================================

enter your choice = 2

Enter your Username = riddhiman
Enter your 4 DIGIT Password = 1298

Login successful!

======================== MAIN MENU ========================
1. CREATE BANK ACCOUNT
2. TRANSACTION
3. CUSTOMER DETAILS
4. TRANSACTION DETAILS
5. DELETE ACCOUNT
6. QUIT
===========================================================

Enter your CHOICE =
```

OUTPUT SCREEN 3: CREATE BANK ACCOUNT

Selecting Option 1 initiates account creation. The operator enters customer account number, full name, phone number, location, and initial credit amount.

```
======================== MAIN MENU ========================
1. CREATE BANK ACCOUNT
2. TRANSACTION
3. CUSTOMER DETAILS
4. TRANSACTION DETAILS
5. DELETE ACCOUNT
6. QUIT
===========================================================

Enter your CHOICE = 1

Enter your ACCOUNT NUMBER = 1001
Enter your ACCOUNT NAME = Rohan Sharma
Enter your PHONE NUMBER = 9876543210
Enter your place = Delhi
Enter your credit amount = 50000

Account Created Successfully!!!!!
```

OUTPUT SCREEN 4: TRANSACTION PROCESSING

Selecting Option 2 presents the transaction processing engine. Below are samples of cash deposit and cash withdrawal executions:

```
-- TRANSACTION EXECUTION: CASH DEPOSIT --
Enter your CHOICE = 2

Enter Your Account Number = 1001

1. WITHDRAW AMOUNT
2. ADD AMOUNT

Enter your CHOICE = 2

Enter amount to be added = 15000

Account Updated Successfully!!!!!

-- TRANSACTION EXECUTION: CASH WITHDRAWAL --
Enter your CHOICE = 2

Enter Your Account Number = 1001

1. WITHDRAW AMOUNT
2. ADD AMOUNT

Enter your CHOICE = 1

Enter withdrawal amount = 5000

Account Updated Successfully!!!!!
```

OUTPUT SCREEN 5: CUSTOMER DETAILS & STATEMENT LOG

Option 3 displays profile information, and Option 4 generates a formatted chronological statement log:

```
-- OPTION 3: CUSTOMER DETAILS LOOKUP --
Enter your CHOICE = 3

Enter your account number = 1001

ACCOUNT NO      = 1001
ACCOUNT NAME    = Rohan Sharma
PHONE NUMBER    = 9876543210
ADDRESS         = Delhi
CURRENT BALANCE = 60000.0

-- OPTION 4: TRANSACTION DETAILS AUDIT LOG --
Enter your CHOICE = 4

Enter your account number = 1001

ACC NO     DATE            WITHDRAWAL      DEPOSIT
-----------------------------------------------------
1001       2026-10-26      0.0             15000.0
1001       2026-10-26      5000.0          0.0
```

OUTPUT SCREEN 6: DELETE ACCOUNT & QUIT

Option 5 terminates customer accounts and removes audit records. Option 6 safely terminates the active session:

```
-- OPTION 5: TERMINATE BANK ACCOUNT --
Enter your CHOICE = 5

DELETE YOUR ACCOUNT
Enter your account number = 1001

ACCOUNT DELETED SUCCESSFULLY

-- OPTION 6: SYSTEM EXIT --
======================== MAIN MENU ========================
1. CREATE BANK ACCOUNT
2. TRANSACTION
3. CUSTOMER DETAILS
4. TRANSACTION DETAILS
5. DELETE ACCOUNT
6. QUIT
===========================================================

Enter your CHOICE = 6

DO YOU WANT TO EXIT (y/n) = y

THANK YOU PLEASE VISIT AGAIN
```

09. TESTING

1. SOFTWARE TESTING OVERVIEW

Software Testing is an empirical investigation conducted to provide stakeholders with information about the quality of the software product under test. It validates and verifies that the software application fulfills technical specifications and handles unexpected input gracefully.

2. BLACK BOX TESTING (SPECIFICATION-BASED TESTING)

Black Box Testing evaluates functionality without reference to internal source code structures or implementation mechanics. Test cases are derived strictly from functional specifications.

Characteristics: Advantages & Relevance to Banking:

- Independence: The tester assesses behavior exactly as a bank teller or customer would experience it.
- Equivalence Partitioning: Inputs are divided into valid classes (e.g. valid 4-digit numeric PINs) and invalid classes (e.g. non-existent account numbers, alphabetic PIN inputs).
- Boundary Value Analysis: Tests focus on threshold values, such as zero-balance withdrawals, empty string passwords, and large credit sums.

3. WHITE BOX TESTING (STRUCTURAL TESTING)

White Box Testing analyzes internal logic paths, control structures, and code execution branches. It ensures that every procedural branch operates reliably under edge conditions.

Techniques: White Box Testing Methodologies Applied:

- Statement Coverage: Ensuring that every line of SQL execution and print statement within main_bank.py and menu.py is executed at least once during test runs.
- Branch / Decision Coverage: Ensuring both True and False branches of every conditional statement (if cur.fetchone() is None, if amt > current_bal) are rigorously exercised.
- Database Exception Verification: Verifying that SQL integrity violations (e.g. duplicate primary key entry) are caught and handled without unceremonious script termination.
- Transaction Integrity: Validating that conn.commit() is consistently called, ensuring atomic persistence.

4. SYSTEM TEST CASES EXECUTION MATRIX

The table below details the formal test cases designed and executed against the Bank Management System, confirming complete functional correctness:

TC ID	Module	Test Description	Test Input	Expected Result	Status
TC01	Auth	Register valid user	riddhiman, 1298	User created successfully	PASS
TC02	Auth	Login with valid PIN	riddhiman, 1298	Login successful; show menu	PASS
TC03	Auth	Login with wrong PIN	riddhiman, 9999	Invalid username/password	PASS
TC04	Account	Create unique account	Acc: 1001, Rs.50000	Account Created Successfully	PASS
TC05	Transact	Deposit valid amount	Acc: 1001, +Rs.15000	Balance updated to 65000	PASS
TC06	Transact	Withdraw valid amount	Acc: 1001, -Rs.5000	Balance updated to 60000	PASS
TC07	Transact	Overdraft withdrawal	Acc: 1001, -Rs.90000	Reject: Insufficient Balance	PASS
TC08	Inquiry	Lookup valid account	Acc: 1001	Display correct customer row	PASS
TC09	Inquiry	Lookup invalid account	Acc: 9999	Invalid Account number	PASS
TC10	Statement	Check audit logs	Acc: 1001	Display all logged transactions	PASS
TC11	Delete	Delete active account	Acc: 1001	Account Deleted Successfully	PASS

10. HARDWARE AND SOFTWARE REQUIREMENTS

1. HARDWARE REQUIREMENTS

The Bank Management System is optimized for high computational efficiency, running smoothly on modest personal computer configurations:

Component	Minimum Requirement	Recommended Specification
Processor	Intel Pentium / AMD Athlon Dual-Core	Intel Core i3 / AMD Ryzen 3 or higher
RAM	512 MB RAM	4 GB RAM or higher
Hard Disk Space	40 GB SATA HDD	256 GB SSD (500 MB free for DB)
Display Monitor	14.1 or 15–17 inch Display (1024x768)	1920x1080 Full HD Monitor
Input Devices	Standard Keyboard and Mouse	Standard QWERTY Keyboard and Mouse
Printer	Optional (for hard-copy statements)	Monochrome Laser Printer (B&W)

2. SOFTWARE REQUIREMENTS

- Operating System: Microsoft Windows 7 / 8 / 10 / 11 (32-bit or 64-bit).
- Python Interpreter: Python 3.12 (or any stable Python 3.7+ release).
- Database Management System: MySQL Community Server 8.0 (or MySQL 5.7).
- Python Database Connector: mysql-connector-python package.
- Development Environment: Python IDLE, VS Code, or Windows PowerShell terminal.

11. INSTALLATION PROCEDURE

To deploy and execute the Bank Management System in a new environment, follow the step-by-step procedure detailed below:

1. Step 1: Install Python Runtime

Download Python 3.12 from python.org. Run the installer and ensure the checkbox 'Add Python.exe to PATH' is selected. Verify installation by running python --version in the terminal.

2. Step 2: Install MySQL Server and Connector

Install MySQL Community Server from dev.mysql.com. Set the root password to manager (or update the credentials in the Python source code). Install the MySQL Python connector by executing:

```
pip install mysql-connector-python
```

3. Step 3: Create the Relational Database

Open the MySQL Command Line Client, authenticate with your root password, and initialize the project database:

```
CREATE DATABASE bank;
USE bank;
```

4. Step 4: Execute Database Table Schema Scripts

Navigate to the source code folder and execute the three DDL setup scripts sequentially to create the tables:

```
python table.py
python user_table.py
python transaction_table.py
```

5. Step 5: Launch the Bank Management Application

Launch the primary system entry file:

```
python main_bank.py
```

Follow the interactive prompts to register a user account, authenticate via PIN, and access the main banking dashboard.

12. BIBLIOGRAPHY

The development, database design, and documentation of this project were guided by the following textbooks, reference materials, and web resources:

TEXTBOOKS & ACADEMIC REFERENCES

- Sumita Arora, Computer Science with Python - Textbook for Class XII, Dhanpat Rai & Co. Publications.
- Preeti Arora, Computer Science with Python for Class XII, Sultan Chand Publications.
- NCERT, Computer Science - Class XII (Python), National Council of Educational Research and Training.
- Praveen M Jigajinni, Project Guidance and Reference Material on Bank Management System (BMS), Sainik School Amaravathinagar.

ONLINE DOCUMENTATION & TECHNICAL REFERENCES

- Python Software Foundation, Official Python 3.12 Documentation: https://docs.python.org/3/
- Oracle Corporation, MySQL 8.0 Reference Manual: https://dev.mysql.com/doc/refman/8.0/en/
- Oracle Corporation, MySQL Connector/Python Developer Guide: https://dev.mysql.com/doc/connector-python/en/
- w3resource, Python MySQL Database Connectivity Tutorials: https://www.w3resource.com/python-exercises/mysql/
- GeeksforGeeks, CBSE Class 12 Computer Science Project Guidelines: https://www.geeksforgeeks.org/school-learning/